Insertion and Deletion in Circular Singly Linked List:

1. Insertion of a node in the beginning using the circular singly linked list

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
void insertAtBeginning(struct Node **head, int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    if (*head == NULL) {
        newNode->next = newNode;
        *head = newNode;
        return;
    }
    struct Node *temp = *head;
    while (temp->next != *head) {
        temp = temp->next;
    }
    newNode->next = *head;
    temp->next = newNode;
    *head = newNode;
}
void display(struct Node *head) {
    if (head == NULL) return;
    struct Node *temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}
int main() {
    struct Node *head = NULL;
    insertAtBeginning(&head, 30);
    insertAtBeginning(&head, 20);
    insertAtBeginning(&head, 10);
```

Output

```
10 -> 20 -> 30 -> (Back to Head)


=== Code Execution Successful ===
```

2. Insertion of a node in the ending using the circular singly linked list

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node{
    int data;
    struct Node *next;
};
void insertAtend(struct Node **head, int data){
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = data;
    if(*head == NULL){
        *head = newNode;
        newNode->next = *head;
        return;
    }
    struct Node *temp = *head;
    while(temp->next != *head){
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->next = *head;
}
void display(struct Node *head){
    struct Node *temp = head;
    if(head == NULL) return;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}
int main(){
    struct Node *head = NULL;
    insertAtend(&head,10);
    insertAtend(&head,20);
    insertAtend(&head,30);
    display(head);
}
```

Output

```
10 20 30

=== Code Execution Successful ===
```

3. Insertion of a node in the given position using the circular single linked list

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
void insertAtPosition(struct Node **head, int data, int pos) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    if (*head == NULL) {
        *head = newNode;
        newNode->next = *head;
        return;
    }
    if (pos == 1) {
        struct Node *temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        newNode->next = *head;
        temp->next = newNode;
        *head = newNode;
        return;
    }
    struct Node *temp = *head;
    for (int i = 1; i < pos - 1 && temp->next != *head; i++) {
        temp = temp->next;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}
void display(struct Node *head) {
    if (head == NULL) return;

    struct Node *temp = head;
    do {
```

Output

```
5 -> 10 -> 15 -> 20 -> (recursive)


=== Code Execution Successful ===
```

main.c

```c
    if (pos == 1) {
        struct Node *temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        newNode->next = *head;
        temp->next = newNode;
        *head = newNode;
        return;
    }
    struct Node *temp = *head;
    for (int i = 1; i < pos - 1 && temp->next != *head; i++) {
        temp = temp->next;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}
void display(struct Node *head) {
    if (head == NULL) return;

    struct Node *temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(recursive)\n");
}
int main() {
    struct Node *head = NULL;
    insertAtPosition(&head, 10, 1);
    insertAtPosition(&head, 20, 2);
    insertAtPosition(&head, 15, 2);
    insertAtPosition(&head, 5, 1);
    display(head);
    return 0;
}
```

Output

```
5 -> 10 -> 15 -> 20 -> (recursive)


=== Code Execution Successful ===
```

4. Deletion of a node in the beginning using the circular linked list

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
void deleteBeginning(struct Node **head) {
    if (*head == NULL) return;
    struct Node *temp = *head;
    struct Node *last = *head;
    if (temp->next == *head) {
        free(temp);
        *head = NULL;
        return;
    }
    while (last->next != *head) {
        last = last->next;
    }
    *head = (*head)->next;
    last->next = *head;
    free(temp);
}
void display(struct Node *head) {
    if (!head) return;
    struct Node *t = head;
    do { printf("%d -> ", t->data); t = t->next; } while (t != head);
    printf("(Head)\n");
}
int main() {
    struct Node *n1 = malloc(sizeof(struct Node));
    struct Node *n2 = malloc(sizeof(struct Node));
    n1->data = 10; n2->data = 20;
    n1->next = n2; n2->next = n1;
```

Output

```
20 -> (Head)


=== Code Execution Successful ===
```

5. Deletion of a node in the ending of circular linked list.

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
void deleteEnd(struct Node **head) {
    if (*head == NULL) return;
    struct Node *temp = *head;
    if (temp->next == *head) {
        free(temp);
        *head = NULL;
        return;
    }
    struct Node *prev = NULL;
    while (temp->next != *head) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = *head;
    free(temp);
}
void display(struct Node *head) {
    if (!head) return;
    struct Node *t = head;
    do { printf("%d -> ", t->data); t = t->next; } while (t != head);
    printf("(Head)\n");
}
int main() {
    struct Node *n1 = malloc(sizeof(struct Node));
    struct Node *n2 = malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
```

Output

```
10 -> (Head)


=== Code Execution Successful ===
```

6. Deletion of a node in the given position of using circular linked list.

Programiz
C Online Compiler

main.c

Share | Run

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
void deleteAtPosition(struct Node **head, int pos) {
    if (*head == NULL) return;
    if (pos == 1) {
        struct Node *temp = *head;
        struct Node *last = *head;
        if (temp->next == *head) {
            free(temp); *head = NULL; return;
        }
        while (last->next != *head) last = last->next;
        *head = (*head)->next;
        last->next = *head;
        free(temp);
        return;
    }
    struct Node *temp = *head;
    struct Node *prev = NULL;
    for (int i = 1; i < pos && temp->next != *head; i++) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = temp->next;
    free(temp);
}
void display(struct Node *head) {
    if (!head) return;
    struct Node *t = head;
    do { printf("%d -> ", t->data); t = t->next; } while (t != head);
```

Output | Clear | Programiz Pl

```
10 -> 20 -> (Head)

=== Code Execution Successful ===
```

main.c

Share | Run

```c
        free(temp);
        return;
    }
    struct Node *temp = *head;
    struct Node *prev = NULL;
    for (int i = 1; i < pos && temp->next != *head; i++) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = temp->next;
    free(temp);
}
void display(struct Node *head) {
    if (!head) return;
    struct Node *t = head;
    do { printf("%d -> ", t->data); t = t->next; } while (t != head);
    printf("(Head)\n");
}
int main() {
    struct Node *n1 = malloc(sizeof(struct Node));
    struct Node *n2 = malloc(sizeof(struct Node));
    struct Node *n3 = malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 15;
    n3->data = 20;
    n1->next = n2;
    n2->next = n3;
    n3->next = n1;
    struct Node *head = n1;
    deleteAtPosition(&head, 2);
    display(head);
    return 0;
}
```

Output | Clear

```
10 -> 20 -> (Head)

=== Code Execution Successful ===
```